CORE103 Computational Thinking
PW8 - Lists



Department of Computer Science

Dhanani School of Science and Engineering

Habib University

Fall 2026

Copyright @ 2026 Habib University

Contents

1 Introduction **2**
1.1 Instructions 2
1.2 Objectives 2

2 Prerequisites **3**
2.1 Assert Statements 3
2.2 Pytest 3

3 Exercises **5**
3.1 A Family Affair 5
3.2 Remove All Occurrences from a List 7
3.3 No Adjacent Repeats 8
3.4 Prime Factors and Special Numbers 10
3.5 Riffle Shuffle 12
3.6 Chain Reaction 13
3.7 Catalan Numbers 15
3.8 The Undiscovered Country 16
3.9 Generatus Emailius - Reloaded 18
3.10 HU GPA Calculator 19

Introduction

1.1 Instructions

- This PW is **UNGRADED**. However, it is highly encouraged that you attempt and submit it to get feedback from your section tutor.
- This PW must be submitted online via CANVAS. You are required to submit a zip file that contains all the `.py` files.
- The zip file should be named as `PW8_aa123456.zip` where `aa123456` will be replaced with your student id.

1.2 Objectives

Following are the objectives of this practice worksheet:

- Master the fundamentals of Python lists, including creation, indexing, and manipulation.
- Apply lists to solve programming challenges, demonstrating proficiency in list-based data manipulation and problem-solving.

Prerequisites

2.1 Assert Statements

Assert statements are crucial tools in programming used to verify assumptions and test conditions within code. For instance, in Python, an assert statement is typically as follows:

```
    assert condition, message
```

For example, in a function calculating the square root, in order to ensure that the input value is non-negative, one might add the following assert statement:

```
assert x >= 0, "x should be a non-negative number"
```

If the condition evaluates to False, an AssertionError is raised, highlighting the issue and its corresponding message.

2.2 Pytest

Pytest is a popular testing framework in Python known for its simplicity and power in facilitating efficient and scalable testing for applications and software. In order to use Pytest, one first needs to install it and create test functions prefixed with *test_* in a Python file. To install Pytest, you can write the following command on the terminal:

```
    pip install pytest
```

After installing Pytest, one can execute all the tests by typing the following command on the terminal:

```
    pytest
```

If you want to test any particular file, then you will have to specify the filename as follows:

```
    pytest <filename>.py
```

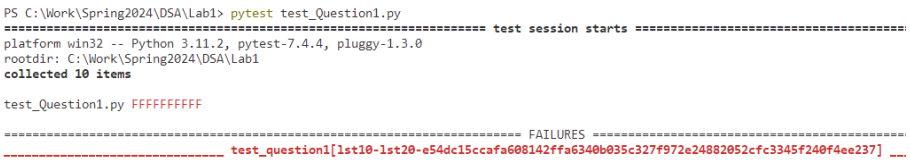
If all the tests pass successfully, then you should receive a similar output.

```
PS C:\Work\Spring2024\DSA\Lab1> pytest test_Question1.py
========================================================================================== test session starts ========
platform win32 -- Python 3.11.2, pytest-7.4.4, pluggy-1.3.0
rootdir: C:\Work\Spring2024\DSA\Lab1
collected 10 items

test_Question1.py ..........

========================================================================================== 10 passed in 0.06s =========
```

However, if all the test fail, then you should receive a similar output.



Exercises

3.1 A Family Affair

This question should be implemented in the file ***q1.py***.

Problem Description

Dr. Crusher has found that L'tak Terai runs in the family. Another victim was Spock's cousin, Selek, though the condition affected him differently. Selek saw digits as letters.



Within a single number, each digit always maps to the same letter. For example, if the digit `9` maps to the letter `c`, then every occurrence of `9` in that number must map to `c`. In another number, `9` may map to a different letter.

Dr. Crusher gives you a list of digits num and a string word. Your task is to determine whether word could be a possible reading of num by Selek.

For example, consider:

```
num = [2, 3, 2, 4, 1]
word = "cacta"
```

The corresponding mapping can be:

Digit	Letter
2	c
3	a
4	t
1	a

Since both occurrences of `2` map to `c`, the mapping is consistent and `"cacta"` is a possible reading.

Function Description

Write a function **selek_sees** that takes a list num and a string word.

The function should return `True` if word is a possible reading of num, and `False` otherwise.

Note

Each value in num will be an integer between `0` and `9`, inclusive. 1. The length of word will be equal to the length of num, and word will be non-empty and contain only lowercase English letters.

Different digits may map to the same letter. However, every occurrence of the same digit within num must map to the same letter.

Sample Interaction

```
>>> selek_sees([2, 3, 2, 4, 1], "cacta")
True

>>> selek_sees([5], "z")
True

>>> selek_sees([7, 9], "ab")
True

>>> selek_sees([1, 2, 2, 1], "xqqr")
False

>>> selek_sees([1, 2, 3, 2, 1], "wwwww")
True
```

Sample Interaction Explanation

In the first case, the corresponding mapping is `2` to `c`, `3` to `a`, `4` to `t`, and `1` to `a`. The repeated digit `2` maps to the same letter both times, so the function returns `True`.

In the second case, the only mapping required is `5` to `z`, so the function returns `True`.

In the third case, `7` maps to `a` and `9` maps to `b`, so the function returns `True`.

In the fourth case, the first `1` maps to `x`, but the second `1` maps to `r`. Since the same digit cannot map to two different letters within the same number, the function returns `False`.

In the fifth case, all digits map to `w`. This is allowed because different digits may map to the same letter. The repeated digits still map consistently, so the function returns `True`.

Testing

In order to test your functions, type one of the following commands in the terminal:

```
python -m pytest tests/test_q1.py
python3 -m pytest tests/test_q1.py
```

3.2 Remove All Occurrences from a List

This question should be implemented in the file ***q2.py***.

Problem Description

In Python, lists have a built-in method called **remove()** that removes the first element whose value is equal to the given argument. If no such value is found, the method raises an error.

Your task is to remove **all occurrences** of a given value from a list while preserving the order of the remaining elements.

Function Description

Write a function **remove_all** that takes a list t and a value v. The function must remove all occurrences of v from the **original list** t.

The function should return the number of items removed. If v does not occur in t, the function should return `0` and leave t unchanged.

Note

- You may not use list indexing, such as **t[5]**, **t[-1]**, or **t[i]**.
- You may not use list slicing, such as **t[2:5]**, **t[i:i + 3]**, or **t[:]**.
- The list may contain elements of any type. Do not use type-specific operations to determine whether an element matches v.
- The order of the remaining elements must be preserved.

Sample Interaction

```
>>> t = [8, 6, 6, 7, 5, 3, 7, 0, 7, 7, 9, 7]
>>> t
[8, 6, 6, 7, 5, 3, 7, 0, 7, 7, 9, 7]

>>> remove_all(t, 2)
0
>>> t
[8, 6, 6, 7, 5, 3, 7, 0, 7, 7, 9, 7]

>>> remove_all(t, 5)
1
>>> t
[8, 6, 6, 7, 3, 7, 0, 7, 7, 9, 7]

>>> remove_all(t, 7)
5
>>> t
[8, 6, 6, 3, 0, 9]
```

Sample Interaction Explanation

In the first case, `2` does not occur in the list, so no element is removed. The function returns `0`, and the list remains unchanged.

In the second case, `5` occurs once. It is removed from the original list, and the function returns `1`.

In the third case, `7` occurs five times. All five occurrences are removed while the order of the remaining elements is preserved. The function therefore returns `5`.

Testing

In order to test your functions, type one of the following commands in the terminal:

```
python -m pytest tests/test_q2.py
python3 -m pytest tests/test_q2.py
```

3.3 No Adjacent Repeats

This question should be implemented in the file ***q3.py****.*

Problem Description

You are given a list of strings called words. For each word, determine the minimum number of character replacements required so that no two adjacent characters are the same.

You may replace any character with any other lowercase character.

Function Description

Write a function **minimum_replacements** that takes a list of strings words and returns a list of integers.

Each value in the returned list should represent the minimum number of replacements required for the word at the corresponding position in words.

Note

- `1 <= len(words) <= 100`.
- Each word will contain between `2` and `100000` characters.
- Each word will contain only lowercase English letters `a`–`z`.

Sample Interaction

```
>>> minimum_replacements(["ab", "aab", "abb", "abab", "abaaaba"])
[0, 1, 1, 0, 1]
```

Sample Interaction Explanation

For `"ab"`, the two adjacent characters are already different, so no replacement is required.

For `"aab"`, one of the adjacent `'a'` characters must be replaced, so `1` replacement is required.

For `"abb"`, one of the adjacent `'b'` characters must be replaced, so `1` replacement is required.

For `"abab"`, no two adjacent characters are the same, so no replacement is required.

For `"abaaaba"`, the consecutive `'a'` characters in the middle can be fixed with one replacement, so `1` replacement is required.

Therefore, the function returns:

```
[0, 1, 1, 0, 1]
```

Testing

In order to test your functions, type one of the following commands in the terminal:

```
python -m pytest tests/test_q3.py
python3 -m pytest tests/test_q3.py
```

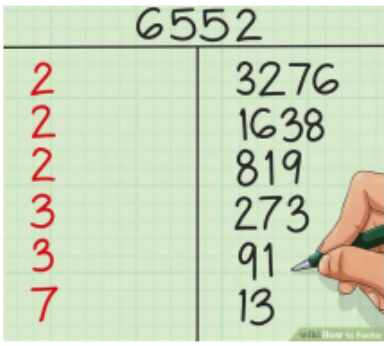
3.4 Prime Factors and Special Numbers

This question should be implemented in the file ***q4.py****.*

Problem Description

A **prime number** is a number greater than 1 that can only be divided evenly by 1 and itself. Every other positive whole number greater than 1 can be broken down into prime factors.

The following image shows the prime factorization of `6552`:



Therefore, the prime factors of `6552` are:

```
[2, 2, 2, 3, 3, 7, 13]
```

A **special number** is a positive integer that is equal to the sum of its prime factors. For example, `4` is special because:

$$4 = 2 + 2$$

Function Description

Write the following functions:

- **prime_factors** takes a positive integer num and returns a list containing all of its prime factors in ascending order.

  Repeated prime factors should appear repeatedly in the returned list.

- **special_numbers** takes a positive integer num and returns a list containing all special numbers from `1` up to and including num.

  You may use your **prime_factors** function to help determine whether a number is special.

Note

The values passed to **prime_factors** and **special_numbers** will be positive integers.

For num = `1`, **prime_factors** should return an empty list because `1` has no prime factors.

Sample Interaction

```
>>> prime_factors(6552)
[2, 2, 2, 3, 3, 7, 13]

>>> prime_factors(1567)
[1567]

>>> prime_factors(1)
[]

>>> special_numbers(25)
[2, 3, 4, 5, 7, 11, 13, 17, 19, 23]
```

Sample Interaction Explanation

For `6552`, the prime factors are `2, 2, 2, 3, 3, 7` and `13`, as illustrated in the image above.

The number `1567` is prime, so its only prime factor is itself and the function returns `[1567]`.

For `1`, there are no prime factors, so **prime_factors(1)** returns an empty list.

For **special_numbers(25)**, each positive integer from `1` through `25` is considered according to the definition of a special number. For example, `4` is included because its prime factors are `[2, 2]` and:

$$2 + 2 = 4$$

Therefore, the function returns:

```
[2, 3, 4, 5, 7, 11, 13, 17, 19, 23]
```

Testing

In order to test your functions, type one of the following commands in the terminal:

```
python -m pytest tests/test_q4.py
python3 -m pytest tests/test_q4.py
```

3.5 Riffle Shuffle

This question should be implemented in the file ***q5.py****.*

Problem Description

A deck of cards can be riffle shuffled by dividing it into two equal piles and interleaving cards from the two piles. In a perfect riffle shuffle, one card is taken from the second pile, followed by one card from the first pile, and this pattern continues until all cards have been placed back into the deck.

In Python, a deck can be represented using a list. For example:

```
[1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
```

is divided into:

```
[1, 2, 3, 4, 5]
[6, 7, 8, 9, 10]
```

Interleaving the two halves produces:

```
[6, 1, 7, 2, 8, 3, 9, 4, 10, 5]
```

Function Description

Write a function **riffle_shuffle** that takes a list t and modifies the **original list** so that its items are arranged according to the riffle shuffle described above.

The function should modify t in place and should not print or return anything.

Note

The list t will always contain an even number of items.

You may not use global variables or import any modules.

Sample Interaction

```
>>> t = [
    'aardvark', 'buffalo', 'crocodile', 'dingo',
    'elephant', 'fox', 'gorilla', 'hippopotamus'
    ]
>>> riffle_shuffle(t)
>>> t
['elephant', 'aardvark', 'fox', 'buffalo',
 'gorilla', 'crocodile', 'hippopotamus', 'dingo']

>>> t = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
>>> riffle_shuffle(t)
>>> t
[6, 1, 7, 2, 8, 3, 9, 4, 10, 5]
```

Sample Interaction Explanation

In the first case, the list is divided into two equal halves:

```
['aardvark', 'buffalo', 'crocodile', 'dingo']

['elephant', 'fox', 'gorilla', 'hippopotamus']
```

The first item is taken from the second half (`'elephant'`), followed by the first item from the first half (`'aardvark'`). Then `'fox'` is followed by `'buffalo'`, and the same pattern continues.

The original list t is therefore modified to:

```
['elephant', 'aardvark', 'fox', 'buffalo',
 'gorilla', 'crocodile', 'hippopotamus', 'dingo']
```

Testing

In order to test your functions, type one of the following commands in the terminal:

```
python -m pytest tests/test_q5.py
python3 -m pytest tests/test_q5.py
```

3.6 Chain Reaction

This question should be implemented in the file q6.py.

Problem Description

In a bubble-popping game, a row of bubbles is represented by a list bubbles, where each integer represents the colour of a bubble. Whenever k or more bubbles of the same colour appear consecutively, the entire group pops and disappears. The remaining bubbles then slide together, which may create a new group that can also pop.

This process continues until no group of k or more adjacent bubbles of the same colour remains.

Function Description

Write a function **pop_bubbles** that takes a list bubbles and an integer k, and returns the list of bubbles remaining after the popping process is complete.

- Always remove the **leftmost** group of at least k matching adjacent bubbles.
- If a group contains more than k bubbles, remove the entire group.
- After a group is removed, the remaining bubbles slide together.
- A newly formed group may also pop.
- Continue until no group can be removed.

Note

Each value in bubbles will be an integer between `0` and `9`. You may assume that `1 <= len(bubbles) <= 1000` and `2 <= k <= 10`. You may not use any modules or the `set` data structure.

Sample Interaction

```
>>> pop_bubbles([1, 3, 3, 3, 2, 2, 1], 3)
[1, 2, 2, 1]

>>> pop_bubbles([1, 2, 2, 2, 1, 1], 3)
[]

>>> pop_bubbles([6, 1, 1, 6, 6, 1, 1, 1, 6, 6, 6], 3)
[6, 1, 1]
```

Sample Interaction Explanation

In the first case, the group `[3, 3, 3]` pops, leaving `[1, 2, 2, 1]`. No group of three or more matching adjacent bubbles remains, so this list is returned.

In the second case, the group `[2, 2, 2]` pops first, leaving `[1, 1, 1]`. The three `1`s now form a new group, so they also pop, leaving an empty list.

In the third case, the middle group `[1, 1, 1]` is the leftmost group that can pop, so it is removed first. This causes the `6`s on either side to join into a group of five, which then pops as well. The remaining list is `[6, 1, 1]`.

Testing

In order to test your functions, type one of the following commands in the terminal:

```
python -m pytest tests/test_q6.py
python3 -m pytest tests/test_q6.py
```

3.7 Catalan Numbers

This question should be implemented in the file ***q7.py****.*

Problem Description

Catalan numbers $\{C_n\}$ form a number sequence that appears in the study of many mathematical objects. The sequence begins with:

$$C_0 = 1$$

To generate the next Catalan number, multiply the first and last previously computed Catalan numbers, then the second and second-to-last, and so on, and add all the products together.

The following table illustrates this process:

n	$C_0, \ldots, C_{n-1}$		C_n
1	1	$C_0 \times C_0 = 1 \times 1$	1
2	1, 1	$C_0 \times C_1 + C_1 \times C_0 = 1 \times 1 + 1 \times 1$	2
3	1, 1, 2	$C_0 \times C_2 + C_1 \times C_1 + C_2 \times C_0 = 1 \times 2 + 1 \times 1 + 2 \times 1$	5
4	1, 1, 2, 5	$C_0 \times C_3 + C_1 \times C_2 + C_2 \times C_1 + C_3 \times C_0 = 1 \times 5 + 1 \times 2 + 2 \times 1 + 5 \times 1$	14
5	1, 1, 2, 5, 14	$C_0 \times C_4 + C_1 \times C_3 + C_2 \times C_2 + C_3 \times C_1 + C_4 \times C_0 = 1 \times 14 + 1 \times 5 + 2 \times 2 + 5 \times 1 + 14 \times 1$	42

Your task is to compute the Catalan numbers from C_j up to, but not including, C_k.

Function Description

Write a function **catalan** that takes two integer parameters, j and k, and returns a list containing the Catalan numbers from C_j up to, but **not including**, C_k.

Note

The values of j and k will satisfy:

$$0 \leq j < k$$

The Catalan number at index k is not included in the returned list.

Sample Interaction

```
>>> catalan(0, 1)
[1]

>>> catalan(2, 8)
[2, 5, 14, 42, 132, 429]
```

Sample Interaction Explanation

In the first case, only C_0 is required. Since $C_0 = 1$, the function returns:

```
[1]
```

In the second case, the required Catalan numbers are from C_2 up to, but not including, C_8. The values up to C_5 are shown in the table above. The next two values are:

$$C_6 = C_0 \times C_5 + C_1 \times C_4 + C_2 \times C_3 + C_3 \times C_2 + C_4 \times C_1 + C_5 \times C_0$$

$$= 1 \times 42 + 1 \times 14 + 2 \times 5 + 5 \times 2 + 14 \times 1 + 42 \times 1 = 132$$

and:

$$C_7 = C_0 \times C_6 + C_1 \times C_5 + C_2 \times C_4 + C_3 \times C_3 + C_4 \times C_2 + C_5 \times C_1 + C_6 \times C_0$$

$$= 1 \times 132 + 1 \times 42 + 2 \times 14 + 5 \times 5 + 14 \times 2 + 42 \times 1 + 132 \times 1 = 429$$

Therefore:

```
  [C2, C3, C4, C5, C6, C7]
= [2, 5, 14, 42, 132, 429]
```

and the function returns:

```
[2, 5, 14, 42, 132, 429]
```

Testing

In order to test your functions, type one of the following commands in the terminal:

```
python -m pytest tests/test_q7.py
python3 -m pytest tests/test_q7.py
```

3.8 The Undiscovered Country

This question should be implemented in the file ***q8.py****.*

Problem Description

Mundungus Fletcher loves solving crossword puzzles, especially those with country names hidden as clues. So much so that he spends a great deal of time every evening scanning the newspaper *Daily Prophet* for country names. He ignores case and non-alphabetic characters in the sentence while searching for country names.

For example, the sentence, "The pleasure**s we den**y ourselves are often the most tempting", contains the country name `"Sweden"` between positions `12` and `20`, as highlighted in the sentence. The positions follow the convention of Python slice notation: the beginning position is included, while the ending position is not included.

Function Description

Write a function **word_search** that takes two arguments, word and sentence, both of type str. The function should return a list containing the beginning and ending positions of the first occurrence of word in sentence.

The search should be case-insensitive. While searching through sentence, non-alphabetic characters should be ignored. The returned positions should refer to positions in the original sentence.

If word does not appear in sentence, the function should return an empty list.

Note

- isinstance(word, str) and len(word) $>$ 0
- isinstance(sentence, str) and len(sentence) $>$ 0
- word will contain alphabetic characters only.

Sample Interaction

```
>>> word_search("Japan", "Aaj apa narangi laayi hain.")
[2, 9]

>>> word_search("Mozambique", "Aaj apa musambi laayi hain.")
[]
```

Sample Interaction Explanation

In the first case, `"Japan"` is found in the sentence when case and non-alphabetic characters in the sentence are ignored. The first matching letter `'j'` is at position `2`, while the final matching letter `'n'` is at position `8`. Following slice notation, the function therefore returns `[2, 9]`.

In the second case, `"Mozambique"` does not appear in the sentence, so the function returns an empty list.

Testing

In order to test your functions, type one of the following commands in the terminal:

```
python -m pytest tests/test_q8.py
python3 -m pytest tests/test_q8.py
```

3.9 Generatus Emailius - Reloaded

This question should be implemented in the file ***q9.py****.*

Problem Description

As an intern at Hogwarts University's Incantation Technology (IT) department, you are given the exciting task of generating email IDs for the incoming batch of students. You have not quite gotten the hang of your chants yet, but luckily your programming-fu is strong. It takes you no time to write a function or two to generate the IDs for you. As far as results go, that is as good a magic as any!

An ID is generated from a name as follows. There is a running serial number starting at `0`. The ID begins with two lowercase letters representing the initial of the first part and the initial of the last part of the student's name. These letters are followed by the serial number written using five digits. The ID ends with `"@st.hu.edu.pk"`. Once an ID is generated, the serial number is incremented for the next ID.

Function Descriptions

You are required to implement the following functions:

- **generate_ids**: The function takes a parameter names, of type list, and returns a list of IDs corresponding to the names in names. The serial number starts at `0` for the first name in the list.
- **main**: The function does not take any parameters. It should read the names from the input, prepare the list names, pass it to **generate_ids**, and print the returned list.

  The input consists of a single line containing a comma-separated list of names. Each name contains at least a non-empty first name and a non-empty last name separated by whitespace. There may also be spaces before or after each name.

Sample Interaction

In the sample below, lines written as [Input -> 'value'] represent values entered by the user while the function is running. The quotation marks are shown only to make spaces in the input visible; they are **not** part of the actual input and are **not** output produced by the function.

```
>>> main()
[Input -> '  Waqar.  Saleem, Umair Azfar   Khan, Shahid Hussain,
     Shah Jamal Alam']
['ws00000@st.hu.edu.pk', 'uk00001@st.hu.edu.pk',
 'sh00002@st.hu.edu.pk', 'sa00003@st.hu.edu.pk']
```

Sample Interaction Explanation

For `"Waqar. Saleem"`, the first initial is `w` and the last initial is `s`. The serial number is `0`, written using five digits as `00000`. Therefore, the corresponding ID is `ws00000@st.hu.edu.pk`.

For `"Umair Azfar Khan"`, the first initial is `u` and the last initial is `k`. The serial number is `1`, written as `00001`. Therefore, the corresponding ID is `uk00001@st.hu.edu.pk`.

The same process is followed for the remaining names.

Testing

In order to test your functions, type one of the following commands in the terminal:

```
python -m pytest tests/test_q9.py
python3 -m pytest tests/test_q9.py
```

3.10 HU GPA Calculator

*This question should be implemented in the file **q10.py**.*

Problem Description

At Habib University, the grade point earned in a course depends on the score received in that course. The following table shows the grade point corresponding to each score range.

Score	Grade Point
$[95, 100]$	4.0
$[90, 95)$	4.0
$[85, 90)$	3.7
$[80, 85)$	3.3
$[75, 80)$	3.0
$[70, 75)$	2.7
$[67, 70)$	2.3
$[63, 67)$	2.0
$[60, 63)$	1.7
$[0, 60)$	0.0

A student's GPA is the weighted average of the grade points earned in all courses, where each course is weighted by its number of credit hours.

For example, suppose the credit hours are:

```
[3, 3, 1, 4, 4]
```

and the corresponding scores are:

```
[80, 73, 94, 62, 85]
```

The corresponding grade points are:

```
[3.3, 2.7, 4.0, 1.7, 3.7]
```

The GPA is:

$$\frac{3(3.3)+3(2.7)+1(4.0)+4(1.7)+4(3.7)}{3+3+1+4+4} = \frac{43.6}{15} \approx 2.91$$

Function Description

Write the following functions:

- **calculate_gpa** takes two lists, credits and scores, and returns the corresponding GPA.

  Entries at the same index in both lists represent the same course. That is, **scores[i]** is the score earned in the course worth **credits[i]** credit hours.

  The function should **return** the GPA rounded to **2 decimal places** and should not print anything.

- **main** reads the credit hours and scores from the user. The first line contains space-separated credit hours, and the second line contains space-separated scores.

  The function should convert the inputs into the appropriate lists, call **calculate_gpa**, and print the GPA returned by the function.

Note

credits and scores will have the same length. Each value in credits will be an integer between `1` and `4`, inclusive, and each value in scores will be between `0` and `100`, inclusive.

Sample Interaction

In the sample below, lines written as [Input -> value] represent values entered by the user while the function is running. They are **not** output produced by the function.

```
>>> main()
[Input -> 3 3 1 4 4]
[Input -> 80 73 94 62 85]
2.91

>>> main()
[Input -> 2 1 1 3 1 3 4]
[Input -> 71 60 97 86 68 66 95]
3.1
```

Sample Interaction Explanation

In the first case, the credit hours are `[3, 3, 1, 4, 4]` and the corresponding grade points are `[3.3, 2.7, 4.0, 1.7, 3.7]`. The total weighted grade points are `43.6`, while the total number of credit hours is `15`. Therefore, the GPA is `43.6 / 15`, which rounds to `2.91`.

In the second case, the scores correspond to the grade points `[2.7, 1.7, 4.0, 3.7, 2.3, 2.0, 4.0]`. Using their corresponding credit hours gives a GPA of `3.1`.

Testing

In order to test your functions, type one of the following commands in the terminal:

```
python -m pytest tests/test_q10.py
python3 -m pytest tests/test_q10.py
```